Lumen Design Constitution

A Foundational Charter for Semantic Infrastructure Research

Lumen Project

Version 0.3 — June 11, 2026

Version: v0.3
Status: Foundational Working Draft (Post-Diagnostic Revision; gate for R5 pre-registrations)
Language: English

Abstract

Lumen is a research project investigating whether structured code representations—specifically those augmented with type annotations and behavioral contracts—affect the accuracy of large language models on code reasoning tasks, including program understanding, bug detection, and code transformation. In the short term, Lumen is a controlled empirical study comparing five progressively richer code representations as inputs to frontier LLMs. In the long term, Lumen aims to develop a *canonical core representation* for programs that is independent of surface syntax, serving as semantic infrastructure on which humans, LLMs, and automated reasoning tools can operate with equal standing.

This document is the project's design constitution: the authoritative record of what has been decided, what remains open, and what principles govern future decisions. It is intended to be a living document, revised as experiments yield evidence and as understanding deepens.

Version 0.3 Changelog (Post-Diagnostic Revision)

Version 0.3 absorbs the outcomes of the R1 measurement-design pass and the R2 analysis-integrity pass (Linear milestones R0–R2) so that all forthcoming confirmatory pre-registrations (R5: T1 and T3) derive from a single, evidence-updated source of truth. **Pre-registrations for the T1 and T3 confirmatory studies derive from this version (v0.3) of the Constitution and the scorers ratified herein.** Each of the five v0.3 decision areas is recorded below, including those resolved as "no change".

1. **Success criteria and primary endpoints.** *Decision.* The pre-registered T2 composite (0–3) is retained *unaltered* as the frozen record of the bug-detection study; the diagnosed ceiling/tie collapse (paper §5.3) is treated as a measurement limitation of that run, not a reason to mutate it. For *future* task families: the T1 primary endpoint is the continuous checklist fraction (`score_t1_checklist`, fraction of ground-truth properties correctly stated, $\in [0, 1]$); the T3 primary endpoint is the continuous test-pass fraction (`score_t3`, passed/total, $\in [0, 1]$, executed in a sandboxed subprocess with timeout). The central comparison remains C4 vs C1+ (H1). For any future T2 re-run, the continuous location metric (`score_t2_continuous`) is designated a *secondary* endpoint reported alongside—not replacing—the frozen composite, until a re-registered T2 study adopts it as co-primary. *Rationale.* The R1 scorers are

continuous-by-construction and were shown (R1-1, R1-3, R1-4, R1-5) to avoid the $\leq$ 3-level saturation that collapsed the T2 effective n; adopting them as the T1/T3 primaries prevents a recurrence while leaving the frozen T2 record intact.

2. **Round-trip validation (C4→C1+→C4′).** *Decision.* Round-trip recovery (R4) is adopted as a *reported diagnostic*, not yet a hard protocol gate: the information-parity (C1+) claim must be accompanied by published recovery rates, but no minimum-recovery threshold is made a pass/fail criterion in v0.3. A threshold may be promoted to a requirement in a later version once a recovery-rate distribution across the dataset is in hand (R4-2). *Rationale.* Setting a numeric threshold before observing the recovery distribution would be arbitrary; transparency (reporting the rates) is required now, calibration of a gate is deferred.

3. **Frozen-script regime and amendment procedure.** *Decision.* The R2-1 bootstrap-CI back-port (`analyze_confirmatory.py --with-ci`; seed 20260427, 10,000 resamples, percentile) is **ratified** as a conforming amendment. Going forward, any change to a frozen analysis script, scorer, or dataset after its freeze date *must* (a) carry a dated in-file amendment note, (b) be accompanied by a standalone amendment document under `docs/`, and (c) leave default (un-flagged) output byte-identical unless the amendment is explicitly a re-freeze. This codifies the procedure whose absence produced the §17.4 gap (an analysis quantity computed by an uncommitted sibling script). *Rationale.* A written, mandatory amendment procedure makes §17.4-class deviations impossible to introduce silently.

4. **Sampling regime under temperature constraints.** *Decision.* Models that reject `temperature=0` are a designed-for case, not an ad-hoc exception. The protocol fixes the deterministic setting (`temperature=0`) as the default; when a model refuses it, the pre-registration must (a) record the minimum permitted temperature actually used, (b) hold it fixed across *all* conditions for that model so the deterministic/stochastic asymmetry never correlates with the representation condition, and (c) report parse-failure rates per condition×model (R2-2) so any temperature×representation confound is visible. Confirmatory comparisons remain *within* a model. *Rationale.* The gpt-5.5/opus-4.7 successor run exposed this gap; holding temperature fixed within a model and disclosing per-cell parse failures keeps the asymmetry from contaminating the C4-vs-C1+ contrast.

5. **Dataset sufficiency / pre-collection power analysis.** *Decision.* A pre-collection power analysis is now **required** before any R5 confirmatory collection: state the assumed effect size, the target power, and the resulting minimum n (functions), and justify the chosen dataset size against it. This closes OQ4 (the Open Questions section) and the Phase-2 gap disclosed in paper §3.0. *Rationale.* The original study set $n = 30$ without a power calculation; the T2 tie-induced effective-n collapse showed the cost of not budgeting power in advance.

Consistency. These decisions are consistent with the Experimental Protocol Addendum section and with the committed R1/R2 artifacts (the `score_t1`, `score_t2_continuous`, `score_t3` modules and the `analyze_confirmatory.py --with-ci` amendment). Where this changelog and older body text differ, the changelog governs for v0.3.

Contents

1 Thesis

Short-Term Thesis (90-Day Horizon)

Structured code representations—ranging from raw abstract syntax trees to contract-augmented intermediate representations—affect the accuracy with which large language models perform code understanding, bug detection, and code transformation tasks. The direction and magnitude of this effect depend on both the representation and the task type. The immediate goal of this project is to quantify these effects through controlled experimentation using Python as the source language and at least two frontier LLMs as subjects.

Long-Term Vision

Programs should have a *canonical* representation that is independent of any particular textual syntax. This canonical core representation would serve as the single source of truth, with textual notations, diagrammatic views, and LLM-optimized serializations treated as projections—derived, potentially lossy renderings of the underlying semantic structure. The long-term vision of Lumen is to develop such a representation and the surrounding infrastructure (editors, verifiers, projection engines) so that humans, LLMs, and formal verification tools can access program meaning on equal terms.

These two goals are related but distinct. The short-term empirical work tests a necessary precondition of the long-term vision: that the form in which code is represented materially affects how well an external agent (here, an LLM) can reason about it. If this precondition fails—if representation form is irrelevant to reasoning quality—then the long-term vision loses a significant part of its motivation, and the project should be redirected accordingly.

2 Scope

In Scope (Next 90 Days)

- Design and execution of a controlled experiment comparing five code representation formats on three LLM-facing tasks.
- Construction of a Python-based pipeline that converts source code into each of the five representations.
- Assembly of an evaluation dataset of 30–50 non-recursive Python functions, covering varied complexity levels and domains.
- A prototype schema for a contract-augmented structured representation (the earliest ancestor of a Lumen core IR).
- A pre-registered statistical analysis with confirmatory and exploratory components.
- A draft workshop paper (4–6 pages, LaTeX).
- Public release of all code, data, prompts, raw outputs, and analysis scripts in a GitHub repository.

Out of Scope (Explicitly Excluded for Now)

- Implementation of an interpreter or compiler for Lumen.

- Formal definition of a type system, effect system, or memory model.
- Design of a projectional editor or structured editing environment.
- Compiler backend targeting (WASM, LLVM, or native).
- Self-hosting or bootstrapping.
- Concurrency or parallelism models.
- Semantic diff algorithms.
- Any claim that Lumen is a usable general-purpose programming language.

3 Non-Negotiables

The following principles govern the project. They are organized into two categories. *Absolute principles* (items 2–3) cannot be overridden. *Long-term research bets* (items 1, 4, 5) are strong architectural preferences that motivate the project's direction; they are not themselves empirical claims and therefore are not directly tested by the 90-day experiment. However, each declares an explicit *evidence threshold* at which the project will formally reassess the bet. This structure resolves the tension between falsifiability (which applies to empirical claims) and architectural commitment (which applies to design direction).

1. **Canonical core over surface text (long-term research bet).** The working assumption is that a structured core IR, not textual source, should be the canonical representation of a program. This assumption shapes the long-term vision but is *not* what the 90-day experiment directly tests. The experiment tests a narrower precondition: whether representation format affects LLM reasoning quality.

   Relationship to the experiment: A positive result (structured representations help) strengthens the motivation for this bet. A negative result (no effect, or text is superior) does not logically refute the bet—the bet may still hold for non-LLM reasons (human cognition, formal verification, tooling)—but it removes the LLM-performance motivation, which is the primary justification offered in this phase.

   Evidence threshold for reassessment: If the experiment shows that $C1^+$ (text with equivalent information) equals or exceeds C4 (structured IR) on all three primary metrics across both LLMs, the project will conduct a documented reassessment of this principle, evaluating whether sufficient non-LLM motivations remain to justify continued pursuit.

2. **Falsifiability (absolute).** Every empirical claim must be testable and refutable. The project will not defend hypotheses against contrary evidence. Long-term research bets (item 1) are not empirical claims and are therefore not subject to direct falsification by a single experiment. However, the *empirical motivations* offered for those bets are falsifiable, and if falsified, the bets lose their stated justification.

3. **Rediscoverability (absolute).** All design decisions, experimental results, and pivots will be documented with their rationale. If this project is abandoned, a future researcher must be able to reconstruct the reasoning from the repository alone.

4. **Staged strictness (long-term research bet).** Any future type system or contract mechanism must support progressive elaboration: zero annotations at the loose end, full dependent

types or contracts at the strict end, with a continuous path between them.

Evidence threshold: If formal analysis demonstrates that staged strictness introduces unsoundness that cannot be repaired without abandoning the principle, it may be replaced by a discrete-level system with explicit boundaries.

5. **LLM-architecture independence (long-term research bet).** The core representation must not depend on the internal architecture of any particular LLM.

   Evidence threshold: If a specific architectural coupling demonstrably improves LLM reasoning accuracy by a large margin ($d > 1.0$) and the coupling can be isolated to the projection layer without contaminating the Core IR, the project may adopt it as a projection-layer optimization.

4 What Is Canonical?

The word *canonical* is central to Lumen and must be used precisely. This section defines what it means in this project and, equally importantly, what it does not yet mean.

Current Definition (90-Day Scope)

Within the current experimental scope, "canonical" refers to **structural canonicalization**: a deterministic normal form for program representations such that two programs that differ only in variable naming (alpha-equivalence) produce the same canonical form.

Concretely:

- Bound variables are converted to de Bruijn indices before canonicalization.
- The canonical form of a single non-recursive function is a directed acyclic graph (DAG) of typed nodes.
- A cryptographic hash is computed over the canonical form. Two structurally equivalent programs (up to alpha-equivalence) yield the same hash.
- The hash of a node is computed over: (node kind, ordered child hashes, type-annotation hash if present, contract hash if present).

Recursion and Cycles

A naive DAG representation cannot express recursive functions or mutually recursive binding groups, which produce cycles in the dependency graph. This is not merely an implementation detail: it directly affects the definitions of canonicality, hashing, and equality. Unison, the closest precedent for content-addressed code, handles this by introducing explicit cycle-aware hash structures for recursive definitions. Lumen must eventually do the same.

90-day scope restriction (hard boundary): The experimental dataset is restricted to *non-recursive, single-function* items. Any function that contains direct self-recursion, mutual recursion via helper functions, or recursive data structures in its signature is **excluded** at the dataset curation stage. This restriction is enforced by a static check in the pipeline (detecting self-references in the AST).

This restriction is acceptable for the 90-day experiment because the experiment tests representation *format* effects, not language completeness. Non-recursive functions provide a sufficient and controlled test bed for this purpose. The restriction is acknowledged as an external validity limitation (see Section 11).

Long-term handling (deferred): Three candidate designs exist and are recorded for future evaluation. None is adopted now.

(a) **Explicit fixed-point wrapper (Unison-style).** Recursive references are wrapped in an explicit `fix` combinator node. The hash of a recursive function f is $H(\texttt{fix}, H(f_{\text{body}}))$, where f_{body} treats the recursive reference as a free variable. This is the approach closest to Unison's content-addressing model.

(b) **Explicit back-edge nodes.** Recursive references are represented as distinguished back-edge nodes pointing to an enclosing binder. The graph remains structurally acyclic with annotated edges.

(c) **Strongly connected component (SCC) grouping.** Mutually recursive bindings are grouped into SCC nodes. Each SCC is hashed as a single unit with a canonical ordering of its members (e.g., lexicographic on de Bruijn-normalized bodies).

The choice among these will be made when the Core IR is extended beyond the experimental prototype, informed by both the Unison precedent and the specific needs of Lumen's projection and verification layers.

What Canonical Does Not Yet Mean

Semantic canonicalization—the property that two programs with identical input–output behavior always produce the same canonical form—is not claimed and not attempted in this phase. Semantic canonicalization is undecidable in general and would require either restricting the language or introducing approximation. It is a long-term research question, not a 90-day deliverable.

Hashing

Hashes are computed on Layer 3 (Core IR) nodes as defined in Section 5. The hash function is provisional (SHA-256 is the default) and may be replaced. The critical property is **determinism**: the same canonical structure must always produce the same hash, regardless of the serialization format used to store or transmit it.

Equality

Two program fragments are *structurally equal in Lumen* if and only if their Layer 3 canonical forms produce identical hashes. This implies alpha-equivalence but not semantic equivalence.

Projection Reversibility

A projection from Layer 3 to a derived form is *lossless* if the original Layer 3 structure can be reconstructed exactly from the projection alone. At least one lossless projection (the reference serialization) must exist at all times. Human-readable and LLM-facing projections may be lossy for readability or token-efficiency reasons, but the lossless serialization must always be available as a fallback.

5 Levels of Representation

Lumen distinguishes four layers of program representation. In the current experimental phase, only Layers 1–3 are implemented, and Layer 3 exists only as a prototype schema.

Layer	Name	Description
4	Serialization	Concrete byte-level encoding of a Layer 3 structure. Multiple serialization formats (JSON, S-expressions, binary) may coexist. Must be fully reversible to Layer 3.
3	Core IR (Canonical)	The canonical representation. A graph of typed nodes with optional type annotations and contracts. Acyclic for non-recursive definitions; cycle handling is deferred (see Section 4). Hashes are computed at this layer. This is the single source of truth in the long-term vision.
2	Projection	Derived views of Layer 3 content, rendered for a specific audience. Includes human-readable text, LLM-optimized serialization, and diagrammatic views. May be lossless or lossy. At least one lossless projection is required.
1	Surface	The user interface layer: editors, IDEs, CLI tools. Edits at this layer are translated into operations on Layer 3 structures. No reversibility requirement.

Experimental Representation Conditions

For the 90-day experiment, five conditions are tested. C1$^+$ is a control condition that isolates the structure-vs.-information confound.

Cond.	Label	Description
C1	Source Text	Raw Python source code, exactly as a developer would read it. Baseline condition.
C1$^+$	Annotated Source Text	Python source code with type annotations inserted inline and behavioral contracts appended as structured docstring comments. Same information content as C4 but in textual form.
C2	Raw AST	The Python abstract syntax tree serialized as JSON, produced by the standard `ast` module. No type information.

Cond.	Label	Description
C3	Typed AST	The AST enriched with type annotations inferred by `mypy`.
C4	Contract-Augmented IR	The typed AST further enriched with behavioral contracts (preconditions, postconditions, invariants). Contracts are generated by an LLM and reviewed by the author per the contract review protocol (Section 17). This is the prototype ancestor of a Lumen Core IR.

The progression from C1 to C4 is intentionally cumulative: each condition adds information to the previous one. $C1^+$ branches off from C4, carrying the same information in textual form, enabling the critical comparison that separates structure from enrichment.

6 Research Questions and Hypotheses

Primary Research Question

RQ: Does the representation format of code—ranging from raw text to contract-augmented structured IR—significantly affect the accuracy of large language models on code understanding, bug detection, and code transformation tasks? In particular, when information content is held approximately constant, does structured representation outperform textual representation?

Confirmatory Hypotheses (Pre-Registered)

These hypotheses define the confirmatory analysis family. All are tested at $\alpha = 0.05$ with Holm–Bonferroni correction applied across the family.

H1 (Structure at constant information)
On at least one primary metric, C4 yields significantly higher accuracy than $C1^+$. This is the **central test** of the Lumen thesis: structured representation helps beyond mere information enrichment.

H2 (Total enrichment)
On at least one primary metric, C4 yields significantly higher accuracy than C1.

H3 (Information enrichment in text)
On at least one primary metric, $C1^+$ yields significantly higher accuracy than C1.

Exploratory Hypotheses (Not Pre-Registered)

The following are examined for pattern discovery but are not part of the confirmatory family. Results are reported with uncorrected p-values and explicitly labeled "exploratory."

E1 (Structure effect without enrichment)
C2 vs. C1: does switching from text to a raw AST (no new information) affect accuracy?

E2 (Type effect)
C3 vs. C2: does adding type annotations to the AST improve accuracy?

E3 (Contract marginal effect)
C4 vs. C3: does adding contracts to the typed AST improve accuracy?

E4 (Task interaction)
The effect of representation format varies across task types.

E5 (Model interaction)
The effect of representation format varies across LLM models.

Null Outcome

If H1 is not rejected (structured representation shows no advantage over annotated text at constant information), the project will document this as a meaningful result. The implication would be that *information content*, not representational structure, is the primary driver of LLM reasoning quality. The long-term Lumen vision would then need to be motivated on grounds other than LLM performance—for example, human factors, formal verification, or tooling benefits.

7 Experimental Framing

Source Language

Python is the sole source language for this experiment. This choice is made for three reasons: (1) Python dominates LLM training corpora, establishing the strongest possible baseline for text-based representation; (2) the `ast` standard library provides reliable parsing; (3) `mypy` provides a mature type inference engine.

The Five Conditions

The experiment uses five conditions. C1 through C4 form a cumulative enrichment chain. $C1^+$ is a **required** information-parity control, not an optional addition.

Why $C1^+$ is required: Without it, the cumulative chain C1→C2→C3→C4 conflates two independent variables: *representation format* (text vs. structured) and *information content* (base vs. type-enriched vs. contract-enriched). Comparisons like C2 vs. C3 or C3 vs. C4, taken alone, measure the effect of *adding information within a structured format*—they do not isolate whether structure itself matters. $C1^+$ holds information content approximately equal to C4 while retaining textual format, enabling the comparison $C1^+$ vs. C4 to isolate the pure structure effect. Without this comparison, the central research claim (that representational *structure* helps LLM reasoning) cannot be supported even if C4 outperforms C1.

$$\text{C1 (text)} \xrightarrow{+\text{structure}} \text{C2 (AST)} \xrightarrow{+\text{types}} \text{C3 (typed AST)} \xrightarrow{+\text{contracts}} \text{C4 (IR)}$$

$$\text{C1 (text)} \xrightarrow{+\text{types, contracts (inline)}} C1^+ \text{ (annotated text)}$$

Key Comparisons and What They Isolate

Comparison	What it isolates
C1 vs. C2	Pure structure effect: does switching from text to an AST (without adding information) help or hurt?
C2 vs. C3	Type-annotation effect within a structured representation.
C3 vs. C4	Contract effect within a structured representation.
C1 vs. C4	Total enrichment effect (endpoint comparison).
C1$^+$ vs. C4	**Structure effect at constant information.** Both carry type and contract information; they differ only in format. This is the critical comparison for the Lumen thesis.
C1 vs. C1$^+$	Information-enrichment effect within textual format (control).

Contract Generation and Review

Contracts for C4 and C1$^+$ are generated by an LLM that is *not* used as a test subject in the experiment. The generated contracts are then reviewed and, if necessary, corrected by the author according to the contract review protocol defined in Section 17. The same contracts (after review) are used in both C1$^+$ and C4, ensuring that information content is held constant across the two conditions.

Separation from the Long-Term Vision

This experiment does not test whether the Lumen canonical Core IR is a good *design for a programming language*. It tests a narrower and more tractable question: whether enriching code representations with structural, type, and contract information improves LLM task performance—and whether the improvement, if any, is attributable to the *format* of the representation or merely to the *additional information* it carries. Positive results on the C1$^+$-vs.-C4 comparison would provide direct evidence for the Lumen thesis. Negative results would redirect the project.

8 Evaluation Rubric

Three task types are used. Metrics are classified as **primary** (used in confirmatory statistical tests) or **secondary** (reported for qualitative analysis only).

T1: Code Understanding

Prompt format: The LLM receives a function in one of the five representations and is asked: "Describe what this function does, including its inputs, outputs, and any important behavioral properties."

Primary metric (auto-scorable checklist): Each function in the dataset has a ground-truth *property checklist*: a list of k verifiable factual properties (e.g., "returns -1 when the input list is empty," "runs in $O(n \log n)$ time"). The score for each item is the fraction of checklist properties correctly stated in the LLM's response (0.0–1.0). Property matching is performed by an automated

classifier (a second LLM or keyword-match script) with spot-check audits by the author on a random 20% sample.

Secondary metric (human-judged holistic score, 1–5): A holistic rubric captures qualities (coherence, completeness of explanation) not fully captured by the checklist. Scored blind: outputs are shuffled and condition labels removed before scoring.

Score	Criterion
5	Correct and complete: all inputs, outputs, edge cases, and behavioral properties accurately described.
4	Correct with minor omissions: core behavior accurate, one edge case or secondary property missing.
3	Partially correct: main purpose identified, but significant behavioral details wrong or missing.
2	Mostly incorrect: main purpose misidentified, though some true details present.
1	Entirely incorrect or incoherent.

Reliability protocol: Because the holistic score is the only human-judged secondary metric retained for T1, its reliability must be formally established.

- The author scores 20% of T1 items twice, separated by at least 48 hours, and reports intra-rater agreement (weighted Cohen's κ).
- If $\kappa < 0.6$, the holistic score is demoted to an appendix-only metric and excluded from all interpretive claims.
- If $\kappa \geq 0.6$, the holistic score remains a secondary metric but is never used in confirmatory analysis.
- The full scoring rubric, with worked examples, is published in the repository appendix to enable future replication.

T2: Bug Detection

Setup: Each function in the dataset has a correct version and a buggy version. Bugs are introduced manually by the author in controlled categories: off-by-one errors, wrong comparison operators, missing boundary checks, incorrect variable references, and swapped arguments. Each bug is annotated with: (a) the exact AST node or line range, (b) the bug category, and (c) a reference fix.

Prompt format: "This function contains exactly one bug. Identify the location of the bug, explain what is wrong, and suggest a fix."

Primary metric (composite score, 0–3, auto-scored):

- **Location** (0 or 1): The LLM identifies the correct line or expression. Automated by substring/AST-node matching against the ground-truth annotation.
- **Diagnosis** (0 or 1): The LLM correctly names the bug category. Automated by keyword matching against a canonical category label set.
- **Fix** (0 or 1): The proposed fix, when applied, passes the function's full test suite. Fully automated.

Secondary metric: None. T2 is fully objective.

T3: Code Transformation

Setup: Each item consists of a function and a natural-language specification of a required change. Each item includes a pre-written test suite for the post-transformation function.

Prompt format: The LLM receives the function (in one of the five representations) plus the change specification and is asked to produce the modified function as executable Python.

Primary metric (test pass rate, auto-scored): The transformed function is executed against the test suite. The score is the fraction of tests passed (0.0–1.0). If the LLM output does not parse or crashes, the score is 0.0.

Dropped metric: The v0.2.1 draft included a "minimal modification" metric (human-judged, binary) for T3. This metric is **dropped** from the main evaluation. Its subjective cost is high relative to its contribution to the research question. If informally interesting patterns are noticed during analysis, they may be noted in the paper's discussion section, but no systematic scoring is performed.

Metric Summary

Task	Metric	Scoring	Role
T1	Property-checklist fraction	Auto + audit	Primary
T1	Holistic 1–5 score	Human, blind	Secondary
T2	Composite 0–3	Auto	Primary
T3	Test pass rate	Auto	Primary

9 MVP and Success Criteria

MVP Definition

The minimum viable product for the first 90 days is:

1. A conversion pipeline (Python scripts) that transforms Python source functions into the five representation conditions (C1, C1$^+$, C2, C3, C4).
2. An evaluation dataset of 30–50 non-recursive Python functions with ground-truth annotations (property checklists for T1, injected bugs and reference fixes for T2, transformation specifications and test suites for T3).
3. Experimental results from at least two frontier LLMs across all five conditions and three task types.
4. A pre-registered statistical analysis (confirmatory + exploratory) with appropriate correction for multiple comparisons.
5. A written report suitable for submission to a workshop venue.
6. A public GitHub repository containing all code, data, prompts, raw LLM outputs, and analysis scripts.

Success Criteria

Success is defined relative to the confirmatory hypotheses (Section 6).

Level	Criterion
Minimum	The experiment completes, all data is collected, and the confirmatory analysis is executed. At least one of H1–H3 is rejected at the corrected significance level. All materials are publicly released.
Moderate	H1 (structure at constant information) is rejected for at least one task type, providing direct evidence for the Lumen thesis. The report characterizes which tasks benefit most from structured representation.
Strong	H1 is rejected for two or more task types. Exploratory analysis reveals an interpretable pattern (e.g., contracts help bug detection more than understanding). The results are sufficient for a workshop paper submission.

Informative Negative Outcomes

If none of H1–H3 is rejected, the project documents this as a meaningful negative result. Specifically: if H1 fails but H3 succeeds, the finding is that information content matters but structure does not, which redirects Lumen toward information-centric designs. If all three fail, representation format is not a first-order factor, and the long-term vision must find alternative motivation.

10 Relation to Prior Work

This section positions the Lumen project relative to existing work. Novelty claims are stated carefully and limited to what the project can actually demonstrate.

Area	Relevant Work	Lumen's Relation
Content addressing	Unison (hash-identified definitions, immutable codebase)	Lumen adopts content addressing as a design principle. Differs in that Unison retains text as the primary authoring surface; Lumen demotes text to a projection.

Area	Relevant Work	Lumen's Relation
Projectional editing	JetBrains MPS, Intentional Programming, Barista	Lumen shares the idea of multiple views over a canonical representation. Differs in explicitly targeting LLMs as a first-class consumer of projections, which prior systems did not consider.
Multi-level IR	MLIR, GraalVM Truffle	These are compiler-infrastructure IRs optimized for transformation passes. Lumen's Core IR serves a different purpose: human–LLM–verifier interoperability, not compiler optimization.
Typed holes / partial programs	Hazel	Hazel supports editing and type-checking incomplete programs. Lumen shares the goal of working with partial information (staged strictness) but extends the audience to LLMs.
Structured prompting for LLMs	Various (chain-of-thought, structured output schemas, code sketches)	Existing work explores ad hoc structuring of LLM inputs. Lumen proposes a principled, language-design-grounded approach to structured representation, tested empirically.
Code representation for LLMs	CodeBERT, GraphCodeBERT, TreeBERT, InCoder	These works embed code structure into model training. Lumen instead modifies the *input representation* presented to a frozen (non-fine-tuned) LLM and measures the effect.

Novelty Claim (Circumscribed)

The novel contribution of the first phase is *not* the individual components (ASTs, type annotations, contracts, or content addressing), all of which exist. The contribution is:

A controlled empirical study measuring how progressively enriched code representations affect the accuracy of frontier LLMs on code reasoning tasks, with an explicit control condition ($C1^+$) that separates the effect of representational *structure* from the effect of information *enrichment*—motivated by and feeding into the design of a canonical core representation for programs.

This framing connects the experiment to the broader vision without overclaiming.

11 Threats to Validity

Threat	**Category**	**Mitigation**
Training-data bias	Construct	LLMs may perform better on text because text dominates training data. Mitigated by testing multiple models, reporting per-model results, and acknowledging this as a standing limitation.
Information confound	Internal	C3/C4 add information absent from C1/C2. The $C1^+$ condition controls for this: $C1^+$ vs. C4 isolates structure at approximately constant information. "Approximately" is acknowledged: minor formatting differences remain.
Small dataset	External	30–50 functions may not generalize. Mitigated by reporting complexity distributions, domain coverage, effect sizes, and confidence intervals alongside p-values.
Contract quality	Internal	LLM-generated contracts may contain residual errors after review. Mitigated by the contract review protocol (Section 17): runtime-checked assertions, structured provenance logging, and a quality rubric.
Evaluator bias	Internal	The sole author scores secondary metrics. Mitigated by: (a) making primary metrics auto-scored, (b) blinding secondary scoring, (c) reporting intra-rater reliability for T1 holistic scores, (d) demoting unreliable metrics.
Prompt sensitivity	Construct	Results may depend on prompt wording. Mitigated by holding prompts constant across conditions (only the representation varies) and publishing all templates in full.

Threat	Category	Mitigation
Multiple testing	Statistical	Five conditions × three tasks × two models produce many possible comparisons. Mitigated by separating confirmatory (3 hypotheses, Holm–Bonferroni corrected) from exploratory analyses (uncorrected, explicitly labeled).
No recursion in dataset	External	Excluding recursive functions limits generalizability. Accepted as a scope limitation for the 90-day study; documented and flagged for future work.

12 Deferred Ambitions

The following are legitimate goals of the Lumen project that are **intentionally excluded** from the 90-day scope. They are recorded here to prevent scope creep and to preserve the long-term roadmap.

1. **Dependent type system.** A formal type system with dependent types is a long-term goal for expressing rich contracts. Deferred until the prototype Core IR is stable.
2. **Algebraic effect system.** Effects as a unified model for IO, errors, and concurrency. Deferred until an interpreter exists.
3. **Memory management model.** Region-based, linear, or garbage-collected memory. Deferred until a compiler backend is under development.
4. **Concurrency model.** Structured concurrency, actors, or other models. Deferred.
5. **Projectional editor.** A structured editor operating directly on Core IR. Deferred until Core IR is stable and at least one lossless projection is implemented.
6. **Semantic diff.** Meaningful diff of Core IR graphs (rename-resistant, etc.). Deferred until content addressing is implemented.
7. **Self-hosting.** Writing Lumen's tools in Lumen. Deferred indefinitely.
8. **Semantic canonicalization.** Normalizing semantically equivalent but structurally different programs to the same form. A deep research problem deferred to a later phase.
9. **General-purpose language status.** Lumen as a language people write production code in. Not a near-term goal.
10. **Recursion and cycle handling in Core IR.** Representing recursive and mutually recursive bindings in the canonical graph. Two candidate designs are recorded in Section 4; selection is deferred until the Core IR extends beyond the experimental prototype.

13 Ninety-Day Plan

The plan assumes 7–10 hours of effort per week, totaling roughly 90–130 hours over 90 days. It is divided into three phases.

Phase 1: Foundations (Days 1–25)

Reading:

- Survey 3–5 papers on structured prompting and code representation for LLMs (LLM4Code, NL4SE workshop proceedings).
- Unison language documentation (content addressing concepts only; no implementation).
- Python `ast` module documentation.
- `mypy` programmatic API for type inference result extraction.

Decisions:

- Finalize the evaluation dataset selection criteria (function length, complexity, domain distribution; non-recursive only).
- Write the formal specification of each representation condition (C1, C1$^+$, C2, C3, C4) as a short internal document.
- Define the T1 property-checklist format and write checklists for the pilot functions.
- Select LLMs for the experiment (default: Claude Sonnet, GPT-4o).
- Select the contract-generation LLM (must differ from test subjects).

Building:

- Python → AST (JSON) conversion pipeline.
- AST → Typed AST pipeline (injecting `mypy` results).
- Typed AST → Contract-Augmented IR pipeline (LLM-generated contracts + author review per Section 17).
- C4 → C1$^+$ projection (inline annotations + docstring contracts).
- Pilot evaluation dataset: 10–15 functions with ground truth.

Publishable:

- GitHub repository with pipeline code and this constitution.

Phase 2: Pilot Experiment and Calibration (Days 26–50)

Reading:

- Statistical methods for within-subjects experimental design (Wilcoxon signed-rank, Holm–Bonferroni, Cohen's d, bootstrap confidence intervals).
- 1–2 exemplar workshop papers from target venues for format and style reference.

Decisions:

- Based on pilot results, calibrate the automated T1 property-checker and T2 scoring scripts.
- Measure intra-rater agreement on T1 holistic scores; decide whether to retain or demote.
- Finalize dataset size (minimum 30, target 50).
- Freeze the analysis script (confirmatory + exploratory).

Building:

- Run pilot experiment on 10–15 functions across all conditions and tasks.
- Build automated scoring scripts for T1 checklist, T2, and T3.
- Complete the full evaluation dataset (30–50 functions).
- Automate the LLM API call and response collection pipeline.

Publishable:

- Experimental protocol document added to the repository.

Phase 3: Full Experiment, Analysis, and Writing (Days 51–90)

Reading:

- Target venue call for papers (for format and deadline alignment).

Decisions:

- Based on full results, determine next-phase direction:
  - If H1 is rejected → proceed to Core IR formal design (Route B).
  - If H1 fails but H3 succeeds → pivot to information-centric design.
  - If all confirmatory hypotheses fail → reassess the canonical-core-first principle per its override threshold.
- Select submission target.

Building:

- Execute full experiment.
- Run confirmatory and exploratory analyses.
- Generate visualizations.
- Draft Core IR v0.01 schema sketch informed by experimental findings.
- Draft workshop paper (4–6 pages, LaTeX).
- Update this Design Constitution to v0.3.

Publishable:

- Complete repository: code, data, prompts, raw outputs, results, analysis scripts.
- arXiv preprint or workshop submission.

14 Repository Starting Structure

```
lumen/
  README.md
  LICENSE
  docs/
    design-constitution.tex
    design-constitution.pdf
    experimental-protocol.md
  src/
    pipeline/
      text_to_ast.py
      ast_to_typed_ast.py
      typed_ast_to_ir.py
      ir_to_annotated_text.py   # C4 -> C1+
      contract_generator.py
      check_no_recursion.py     # static exclusion check
    experiment/
      run_experiment.py
      score_t1_checklist.py
      score_t1_holistic.py
      score_t2.py
      score_t3.py
      analyze_confirmatory.py
      analyze_exploratory.py
    utils/
      llm_client.py
      hash.py
  data/
    functions/
      raw/              # original Python functions
      ast/              # C2: AST JSON
      typed_ast/        # C3: typed AST JSON
      ir/               # C4: contract-augmented IR
      annotated_text/   # C1+: annotated source text
    contracts/
      raw/              # contract_raw.json per function
      reviewed/         # contract_reviewed.json per function
      diffs/            # contract_diff.json per function
    ground_truth/
      checklists/       # T1 property checklists
      bugs/             # T2 bug annotations
      tests/            # T3 test suites
  results/
    raw/                # LLM outputs
    analysis/           # statistical summaries
    figures/            # plots
  paper/
```

```
main.tex
figures/
```

15 Decision Log

Version	Decision	Rationale
v0.1	Lumen adopts Core IR as the canonical representation; text is a projection.	Founding principle. Distinguishes Lumen from all text-first language projects.
v0.1	First target domain is LLM output verification.	Aligns the short-term work with the growing need for machine-generated code quality assurance.
v0.2	90-day route is empirical (Route A), not verification-core design (Route B).	The author is new to PL implementation; an empirical study is feasible within skill set and time budget. Route B is deferred, not abandoned.
v0.2	Source language for experiments is Python.	Maximizes LLM baseline accuracy (most training data) and tooling availability (`ast`, `mypy`).
v0.2	Contracts for C4 are LLM-generated, then author-reviewed.	Balances workload (manual writing of 30–50 contracts is excessive) with quality (unreviewed contracts add noise).
v0.2	"Canonical" is restricted to structural canonicalization (alpha-equivalence via de Bruijn indices).	Semantic canonicalization is undecidable in general and out of scope for 90 days.
v0.2	At least two LLMs will be used in the experiment.	Single-model results cannot distinguish representation effects from model-specific biases.
v0.2	All outputs are in English and LaTeX.	Required for international venue submission and for attracting future collaborators.
v0.2.1	Non-negotiable 1 reclassified as "architectural default" with explicit override threshold.	Review identified tension between unfalsifiable architectural commitment and the falsifiability principle.
v0.2.1	Dataset restricted to non-recursive functions; recursion/cycle handling deferred.	The DAG assumption breaks under recursion. Two candidate designs are recorded but not adopted. The experiment does not require recursive code.

Version	Decision	Rationale
v0.2.1	Fifth condition $C1^+$ added to separate structure from information enrichment.	Review identified that cumulative C1–C4 chain conflates format and information content. $C1^+$ vs. C4 is now the central test.
v0.2.1	Primary metrics made auto-scorable; human-judged metrics demoted to secondary.	Review identified over-reliance on subjective judgment. Confirmatory tests now use only auto-scored metrics.
v0.2.1	Confirmatory / exploratory analysis distinction introduced with Holm–Bonferroni correction.	Review identified under-specified statistical plan and risk of undisciplined multiple testing.
v0.2.1	Contract review protocol formalized with allowed edits, rejection criteria, and provenance logging.	Review identified reproducibility risk in under-specified review process.
v0.2.2	Non-negotiable 1 further refined: reclassified from "architectural default" to "long-term research bet" with explicit relationship to experiment outcomes.	Second review found that the v0.2.1 override-threshold language still appeared to shield the principle from falsification. New formulation separates the bet itself (not falsifiable by one experiment) from its empirical motivation (falsifiable).
v0.2.2	$C1^+$ elevated from "control condition" to "required information-parity control."	Review argued this is not optional but structurally necessary to support the central claim about representation format.
v0.2.2	Recursion/cycle section strengthened: Unison precedent acknowledged; static exclusion check added to pipeline; third candidate design (fixed-point wrapper) recorded.	Review identified that ignoring the Unison cycle-handling precedent weakens credibility of the content-addressing claim.
v0.2.2	T3 minimal-modification metric dropped.	Review judged subjective cost too high relative to diagnostic value for the primary research question.

Version	Decision	Rationale
v0.2.2	T2 (bug detection) designated as primary endpoint within the confirmatory family.	Most objective scoring and most direct relevance to the LLM-output-verification motivation.
v0.2.2	Statistical plan strengthened: primary endpoint designated, per-model analysis classified as exploratory, mixed-effects alternative noted, analysis-script freeze protocol added.	Review identified under-specification of primary vs. exploratory analyses and risk of undisciplined pairwise testing.

16 Open Questions

1. **What contract language should C4 use?** Options include Python-like assertions, a JSON schema of pre/postconditions, or a custom notation. The choice affects both LLM comprehension and the future Core IR design. The same notation must be used for C1$^+$ docstring contracts to maintain information parity.
2. **Which workshop venues are viable targets?** Candidates include LLM4Code (ICSE), MAPS (SPLASH), NL-SE, and ASE Tool Demo. Deadlines and page limits must be checked.
3. **How should the T1 automated property-checker be implemented?** Options: keyword/regex matching, a second LLM as classifier, or NLI-based entailment checking. To be decided during Phase 1 based on pilot feasibility.
4. **Is the dataset large enough for the confirmatory family?** With 9 confirmatory tests and Holm–Bonferroni correction, the effective per-test alpha is reduced. A pilot power analysis (or at minimum, an effect-size sanity check from the pilot data) should be conducted during Phase 2 to determine whether 30 functions suffice or 50 are needed.
5. **Should a mixed-effects model replace or supplement the Wilcoxon approach?** The pilot will reveal whether the simpler nonparametric approach is adequate or whether the repeated-measures structure demands a regression model. Decision deferred to Phase 2.
6. **How should the long-term novelty of Lumen be positioned relative to Hazel's recent LLM work?** Hazel is beginning to explore typed holes in conjunction with LLM assistance. The overlap with Lumen's long-term vision should be tracked and the differentiation sharpened before any long-term vision paper is attempted.

17 Experimental Protocol Addendum

This section consolidates the experimental protocol into a single reference. It is binding for the 90-day study and supersedes any conflicting informal descriptions elsewhere in this document.

17.1 Control Conditions

Condition Table (Definitive)

Cond.	Label	Format	Info Level	Purpose
C1	Source Text	Text	Base	Baseline
$C1^+$	Annotated Text	Text	Enriched	Information-enrichment control
C2	Raw AST	Structured	Base	Pure structure effect (no extra info)
C3	Typed AST	Structured	+Types	Marginal type-annotation effect
C4	Contract IR	Structured	Enriched	Full enrichment in structured form

The critical pair is **$C1^+$ vs. C4**: both carry the same information (types + contracts); they differ only in format (text vs. structured IR).

Information-Parity Guarantee

$C1^+$ is constructed from C4 by projecting its type and contract annotations back into the Python source as:

- PEP 484-style inline type annotations, and
- a structured docstring block containing preconditions, postconditions, and invariants in a fixed template.

Because $C1^+$ is *derived from* C4's annotations, the two conditions carry the same semantic content by construction. Minor formatting differences (e.g., JSON key names vs. docstring labels) are an acknowledged residual confound.

17.2 Contract Review Protocol

Generation

Contracts are generated by a designated *contract-generation LLM* that is **not** used as a test subject. The generation prompt requests preconditions, postconditions, and invariants in a fixed JSON schema.

Allowed Author Edits

During review the author may:

1. **Correct** a factually wrong predicate (e.g., change `len(result) >= 0` to `len(result) == len(input)`).
2. **Delete** a contract clause that is vacuously true, untestable, or redundant.
3. **Add** a missing clause if a critical behavioral property is absent, limited to one added clause per function.

The author may **not**:

- Rewrite the contract from scratch.
- Add domain knowledge that could not reasonably be inferred from the function's signature and body.
- Introduce information not derivable from the source code.

Rejection Criteria

A generated contract is **rejected** (and regenerated with a fresh prompt) if:

- More than 50% of its clauses are factually incorrect.
- It misidentifies the function's return type.
- It contains no postconditions.

At most two regeneration attempts are allowed. If the third attempt also fails, the function is excluded from the dataset and the exclusion is logged.

Provenance Logging

For every function, the repository stores:

1. The raw LLM-generated contract (`contract_raw.json`).
2. The reviewed contract (`contract_reviewed.json`).
3. A diff file (`contract_diff.json`) recording every edit with a one-line justification.
4. The number of regeneration attempts (0, 1, or 2).

Contract Quality Rubric (Minimal)

After review, every contract must satisfy:

Criterion	Requirement
Soundness	Every precondition and postcondition is factually correct for the function as written.
Non-triviality	At least one postcondition is non-vacuous (i.e., it rules out at least one logically possible return value).
Runtime-checkable	Every clause can be expressed as a Python assertion that either passes or raises `AssertionError` on concrete inputs.
Tested	The reviewed contract is executed as runtime assertions on the function's test suite; all assertions pass.

17.3 Scoring Protocol

Primary Metrics (Confirmatory)

Task	Metric	Procedure
T1	Property-checklist fraction	Automated classifier checks each ground-truth property against the LLM response. Author audits a random 20% sample. If audit disagreement > 10%, the classifier is recalibrated and the full set is re-scored.
T2	Composite 0–3	Location, diagnosis, and fix scored automatically against ground-truth annotations. Fix correctness verified by running patched code against the test suite.
T3	Test pass rate	LLM output is parsed as Python. If parsing fails, score = 0. Otherwise, the modified function is executed against the post-transformation test suite. Score = fraction of tests passed.

Secondary Metrics (Exploratory)

Task	Metric	Procedure
T1	Holistic 1–5	Human-judged, blind. Intra-rater reliability reported; demoted if $\kappa < 0.6$. Rubric with worked examples published in repository appendix.

T3 "minimal modification" was included in v0.2.1 but is **dropped** in this revision. Its subjective cost outweighs its diagnostic value for the primary research question.

17.4 Statistical Analysis Plan

Design

Within-subjects (repeated-measures) design: every function is presented to every LLM in every condition. The unit of observation is a (function, condition, model) triple. The design is fully crossed: $N_{\text{functions}} \times 5_{\text{conditions}} \times 2_{\text{models}} \times 3_{\text{tasks}}$.

Primary Endpoint

The single most important test in the experiment is:

> **H1 on T2 (bug detection composite score):** Does C4 yield a higher median composite score than C1$^+$ on the bug detection task?

T2 is designated as the primary endpoint because it has the most objective scoring (fully automated, 0–3 composite) and the most direct relevance to the LLM-output-verification motivation. H1 on T1 and T3 are also confirmatory but are secondary to this test in interpretive weight.

Confirmatory Analysis

Three pre-registered hypotheses (H1, H2, H3) are tested on all three primary metrics, yielding a confirmatory family of 9 tests (3 hypotheses $\times$ 3 primary metrics).

For each test:

1. Compute the paired difference in scores between the two conditions (e.g., C4 minus C1$^+$ for H1), aggregated per function across models (i.e., average the score for model A and model B for each function, then test the paired differences across functions).
2. Test whether the median paired difference is significantly different from zero using the **Wilcoxon signed-rank test** (chosen because scores are ordinal or bounded continuous and normality is not assumed).
3. Report the test statistic, exact p-value, and effect size (rank-biserial correlation r).

Holm–Bonferroni sequential correction is applied across the 9-test family at $\alpha = 0.05$.

Per-Model Analysis

If the two LLMs show qualitatively different patterns (e.g., H1 rejected for model A but not model B), this is reported as a model interaction finding. Per-model results are presented in a supplementary table but are classified as **exploratory**; they are not part of the confirmatory family and do not receive multiplicity correction.

Alternative Analysis (Considered but Deferred)

A mixed-effects model (e.g., ordinal mixed-effects regression with function as random intercept and condition $\times$ task $\times$ model as fixed effects) would more naturally handle the repeated-measures structure and interaction terms. This approach is noted as a candidate for a revised analysis if the dataset is large enough ($N \geq 40$ functions) and if the pilot reveals that the simpler Wilcoxon approach is underpowered. If adopted, it will be documented as a protocol amendment before the full experiment begins.

Exploratory Analysis

All other comparisons (E1–E5, per-task breakdowns, per-model interactions, condition $\times$ task interaction patterns) are reported with uncorrected p-values and explicitly labeled EXPLORATORY. Effect sizes and confidence intervals are always reported. No causal or confirmatory claims are made from exploratory results.

Reporting Conventions

- All p-values are reported to three decimal places.
- Effect sizes are reported with 95% bootstrap confidence intervals (10,000 resamples).
- Raw data and analysis scripts are published in the repository.
- The analysis script is frozen after the pilot and before the full experiment begins. The freeze date is recorded in the repository commit history. Any post-hoc deviations from the frozen script are documented as protocol amendments with justification.

17.5 Handling of Recursion and Cycles

90-day scope: The evaluation dataset includes only non-recursive, single-function items. Functions that call themselves (directly or via mutual recursion) are excluded at the dataset curation stage.

Rationale: The experiment tests the effect of *representation format*, not language completeness. Non-recursive functions provide a controlled test bed while avoiding the unresolved question of how to canonicalize recursive definitions (see Section 4).

Scope limitation: This restriction means the results do not generalize to recursive code. This is documented as an external validity limitation and flagged for future work.

17.6 What Is Explicitly Out of Scope for This Protocol

The following are **not** part of the 90-day experimental protocol:

- Multi-file or multi-function programs.
- Class-level or module-level analysis.
- Recursive or mutually recursive functions.
- Fine-tuning or training LLMs on structured representations.
- Measuring LLM latency, token usage, or cost.
- Evaluating human (non-LLM) performance on the same tasks.
- Any claim about Lumen as a usable programming language.
- Formal verification or proof-carrying code.

These exclusions are intentional scope boundaries, not oversights. Each is a candidate for future work.

Glossary

Canonical Core IR
: The single authoritative representation of a program in Lumen. A graph of typed nodes, identified by content hashes. Acyclic for non-recursive definitions; cycle handling is deferred. In the current phase, canonicalization is structural (alpha-equivalence only).

Projection
: A rendering of the Core IR into a specific format for a specific audience. Text, diagrams, and LLM-optimized serializations are all projections. A projection is *lossless* if the Core IR can be exactly reconstructed from it, and *lossy* otherwise.

Human Projection
: A projection optimized for human reading and writing. Typically a textual notation resembling a conventional programming language.

LLM Projection
: A projection optimized for consumption by a large language model. Its design is an open research question; the 90-day experiment compares candidate formats.

Content Addressing
: Identifying code units by the cryptographic hash of their canonical form rather

than by name or file path. Names are aliases for hashes. Adopted from the Unison language.

Structural Canonicalization
Reducing a program representation to a normal form in which alpha-equivalent programs are identical. Achieved by converting bound variables to de Bruijn indices.

Semantic Canonicalization
The (unachieved, long-term) goal of reducing semantically equivalent programs to identical normal forms. Undecidable in general.

Staged Strictness
The principle that a Lumen program can range from fully untyped to fully dependently typed, with the level of strictness chosen per module. An architectural default of the project.

Contract A formal specification of a function's expected behavior, expressed as preconditions, postconditions, and invariants. In the current experimental phase, contracts are generated by an LLM and reviewed by the author per the contract review protocol.

Semantic Graph Synonym for Core IR when emphasizing its graph structure.

Semantic Diff A diff algorithm operating on Core IR graphs rather than text lines. Resistant to renames and formatting changes. Deferred.

Lossless Projection
A projection from which the original Core IR can be exactly reconstructed. At least one must exist at all times.

Route A The empirical research path: experiments comparing code representations for LLM tasks. The current 90-day focus.

Route B The verification-core research path: formal design of the Core IR with dependent types, contracts, and verification infrastructure. Deferred to a future phase.

Architectural Default
Term used in v0.2.1; superseded by "Long-Term Research Bet" in v0.2.2.

Long-Term Research Bet
A design principle that the project adopts as a strong directional commitment. It is not an empirical claim and is therefore not directly falsifiable by a single experiment. However, it must declare an evidence threshold at which the project will formally reassess the bet and its motivating rationale. Replaces "architectural default" from v0.2.1.

Information-Parity Control
The experimental condition $C1^+$, which carries the same information as C4 (types and contracts) but in textual rather than structured form. Required to separate the effect of representation *structure* from the effect of information *enrichment*.

Primary Endpoint

The single most important test in the confirmatory analysis family: H1 tested on T2 (bug detection composite score). Other confirmatory tests are also reported but carry less interpretive weight.

Confirmatory Analysis

Pre-registered statistical tests (H1–H3) with family-wise error correction (Holm–Bonferroni). Distinguished from exploratory analysis.

Exploratory Analysis

Post-hoc or non-pre-registered statistical analyses, reported with uncorrected p-values and explicitly labeled as exploratory. No causal or confirmatory claims are made from these results.